

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Scenario

The e-commerce database needs to organize product information according to different categories. The system should maintain the details of each product and establish a proper connection between products and their respective categories. This makes it easier to manage products, update their information, and generate useful category-based reports.

1. Design Product and Category Tables

Category Table:

The Category table is designed to maintain the categories available in the e-commerce system. Each category is assigned a unique identifier so that it can be distinguished from other categories.

```
SQL> CREATE TABLE CATEGORY(Category_ID VARCHAR(7),Category_Name VARCHAR(20),Description VARCHAR(50));  
Table created.
```

Product Table:

The Product table stores the details of products available on the platform. It maintains important information required for identifying and managing each product.

```
SQL> CREATE TABLE PRODUCT(Product_ID VARCHAR(7),Product_Name VARCHAR(20),Category_ID VARCHAR(7),Price NUMBER(9,2),Stock_Quantity NUMBER);  
Table created.
```

The Category and Product tables are related because every product is associated with a particular category.

2. Define Relationships and Constraints

Relationships:

The relationship between the Category and Product tables is **One-to-Many (1 : *)**.

This means that one category can be associated with multiple products, whereas a product belongs to one category.

Primary Key: A Primary Key provides a unique identity to every record in a table. It prevents two records from having the same identifying value.

In the Category table, CATEGORY_ID is used to identify each category. In the Product table, PRODUCT_ID uniquely identifies each product.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);  
Table altered.
```

Foreign Key: A Foreign Key is used to establish a connection between related tables. In the Product table, CATEGORY_ID connects each product with the appropriate record in the Category table.

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);  
Table altered.
```

NOT NULL Constraint: The NOT NULL constraint ensures that essential columns cannot be left without a value. This is useful for fields where information is compulsory, such as product or category names.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

CHECK Constraint: The CHECK constraint is used to restrict the values entered into a column according to a specified condition. It helps maintain valid data in the database.

For example, a product price can be restricted to positive values.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (Price > 0);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE Constraint: The UNIQUE constraint prevents duplicate values from being stored in a column. It is useful when a particular value should occur only once in the table.

For example, category names can be maintained without duplicate entries.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);  
Table altered.
```

3. Insert Values

After creating the Category and Product tables and applying the required constraints, records are inserted into the tables.

The INSERT statement is used to add new category and product information to the database. This provides the initial data required for further operations and analysis.

Category Table:

```
SQL> INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description');
Enter value for category_id: 01
Enter value for category_name: Electronics
Enter value for description: Electronic devices and accessories
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('01','Electronics','Electronic devices and accessories')

1 row created.

SQL> /
Enter value for category_id: 02
Enter value for category_name: Clothing
Enter value for description: Men and women clothing products
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('02','Clothing','Men and women clothing products')

1 row created.

SQL> /
Enter value for category_id: 03
Enter value for category_name: Furniture
Enter value for description: Home and office furniture
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('03','Furniture','Home and office furniture')

1 row created.

SQL> /
Enter value for category_id: 04
Enter value for category_name: Books
Enter value for description: Educational and general books
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('04','Books','Educational and general books')

1 row created.

SQL> /
Enter value for category_id: 05
Enter value for category_name: Sports
Enter value for description: Sports equipment and accessories
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
```

Product Table:

```
SQL> INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity);
Enter value for product_id: 101
Enter value for product_name: Laptop
Enter value for category_id: 01
Enter value for price: 55000
Enter value for stock_quantity: 20
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('101','Laptop','01',55000,20)

1 row created.

SQL> /
Enter value for product_id: 102
Enter value for product_name: Smartphone
Enter value for category_id: 01
Enter value for price: 25000
Enter value for stock_quantity: 35
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('102','Smartphone','01',25000,35)

1 row created.

SQL> /
Enter value for product_id: 103
Enter value for product_name: T-Shirt
Enter value for category_id: 02
Enter value for price: 799
Enter value for stock_quantity: 50
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('103','T-Shirt','02',799,50)

1 row created.

SQL> /
Enter value for product_id: 104
Enter value for product_name: Office Chair
Enter value for category_id: 03
Enter value for price: 4500
Enter value for stock_quantity: 15
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('104','Office Chair','03',4500,15)
```

After inserting the records, the contents of both tables can be displayed to verify that the data has been stored successfully.

```
SQL> SELECT * FROM Category;
```

CATEGOR	CATEGORY_NAME	DESCRIPTION
01	Electronics	Electronic devices and accessories
02	Clothing	Men and women clothing products
03	Furniture	Home and office furniture
04	Books	Educational and general books
05	Sports	Sports equipment and accessories

```
SQL> SELECT * FROM Product;
```

PRODUCT	PRODUCT NAME	CATEGOR	PRICE	STOCK	QUANTITY
101	Laptop	01	55000.00		20
102	Smartphone	01	25000.00		35
103	T-Shirt	02	799.00		50
104	Office Chair	03	4500.00		15
105	Programming Book	04	999.00		30

```
SQL> |
```

4. CRUD Operations

CRUD refers to the basic operations performed on database records: **Create, Read, Update, and Delete**. These operations allow the database administrator or application to manage product information throughout its lifecycle.

Insert Record: The Insert operation is used to add a new product record to the Product table. It allows newly available products to be included in the database.

```
SQL> INSERT INTO Product (PRODUCT_ID, PRODUCT_NAME, CATEGORY_ID, PRICE, STOCK_QUANTITY) VALUES ('106', 'Football', '05', 1200, 25);
1 row created.
```

Update Price: The Update operation is used to modify existing product information. In this case, it can be used to change the price of a product when its selling price is revised.

```
SQL> UPDATE Product SET Price = 70000 WHERE Product_Name = 'Laptop';
1 row updated.
```

Delete a Product: The Delete operation removes an unwanted product record from the Product table. It can be used when a product is no longer available or needs to be removed from the database.

```
SQL> DELETE FROM Product WHERE Product_Name = 'Smartphone';
1 row deleted.
```

5. Generate Category-wise Product Reports

- **Display all products under each category:**

This query is used to display the products along with their corresponding categories. It provides a clear view of which products are associated with each category.

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price,p.Stock_Quantity FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID;
```

CATEGORY_NAME	PRODUCT NAME	PRICE	STOCK QUANTITY
Electronics	Laptop	70000.00	20
Clothing	T-Shirt	799.00	50
Furniture	Office Chair	4500.00	15
Books	Programming Book	999.00	30
Sports	Football	1200.00	25

- **Count number of products in each category:**

This query determines the number of products available in every category. The result helps identify categories that contain a larger or smaller number of products.

```
SQL> SELECT c.Category_Name,COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
Clothing	1
Books	1
Electronics	1
Furniture	1
Sports	1

- **Find highest-priced product in each category:**

This query identifies the product with the highest price within each category. It can be used to compare the maximum-priced products across different categories.

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price AS Highest_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID WHERE p.Price =(SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME	PRODUCT NAME	HIGHEST_PRICE
Electronics	Laptop	70000
Clothing	T-Shirt	799
Furniture	Office Chair	4500
Books	Programming Book	999
Sports	Football	1200

- **Display categories having more than 5 products:**

This query displays only those categories whose product count is greater than two. It helps identify categories with a relatively large collection of products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID) > 2;
```

no rows selected

- **Find average product price category-wise:**

This query calculates the average price of products belonging to each category. The result gives an overview of the general price range of products within each category.

```
SQL> SELECT c.Category_Name,AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
Clothing	799
Books	999
Electronics	78000
Furniture	4500
Sports	1200

```
SQL>
```

Conclusion:

The Product and Category Management System provides a structured way to organize e-commerce product information. By establishing relationships and applying appropriate constraints, the database maintains data consistency. CRUD operations make it possible to manage product records, while category-wise SQL queries provide useful information for analyzing products and their prices.